

重庆邮电大学期末试卷

学号：

一、不定项选择题（20）

1.UML 提供的图示中，哪一种最适合用来描述系统随时间变化的动态行为，并清晰展示对象间消息交换的顺序 ？

A. 类图

B. 对象图

C. 组件图

D. 序列图（时序图）

1、 ☐ A ☐ B ☐ C ☐

- 2.关于需求工程中面谈结构的组织，以下哪些描述是正确的 ？
- A. 矩阵结构是面谈中最基础的组织形式
- B. 漏斗结构通常从宽泛的、开放式的问题开始，逐渐收窄到具体细节
- C. 金字塔结构适用于受访者需要通过具体细节引导才能进入状态的情况
- D. 所有面谈结构都必须以闭合式问题（只能回答 Yes/No）作为结束

2、 ☐ A ☐ ☐ ☐ D

3.在需求获取的面谈（Interview）中，如果面试官先询问具体的细节问题，再逐步过渡到概括性的宏观问题，最后以一个总结性的问题结束。这种面谈结构属于 ：

A. 漏斗结构

B. 金字塔结构

C. 菱形结构

D. 线性结构

3、 ☐ A ☐ B ☐ C ☐

4.UML 建模分为静态建模和动态建模。以下哪些图属于描述系统动态行为的行为图 ？

A. 用例图

- B. 类图
- C. 状态机图
- D. 活动图

4、 ☐ ☐ B ☐ ☐ D

二、判断题（10）

在进行需求面谈时，漏斗结构（Funnel Structure）是指面试官首先询问非常具体的问题，随后逐渐引向更宽泛的概括性问题。

5 ☐ ☐ ×

在 UML 建模中，状态机图（State Machine Diagram）主要用于描述一个特定对象在其生命周期内，由于响应事件而经历的状态序列及转换。

6、 ☐ √ ☐

三、填空题（10）

能力成熟度模型集成（CMMI）是衡量软件组织过程能力成熟度的重要标准，它将软件过程的成熟度从低到高划分为 5 个等级，分别是：(1)、(2)、(3)、(4) 以及 (5)_____。

7、 已管理级， 初始级， 持续优化级， 已量化级， 已定义级

四、简答题（10）

8、 简述需求面谈的三种结构（5）

1.金字塔型 2.漏斗型 3.菱形形

9、UML 建模有些什么图（5）

UML图有很多种,常用的包括类图,用例图,序列图,状态图和活动图

五、综合题（50）

1.编程（20）

在图书馆管理系统中，需要定义一个“图书”类（Book）。请使用 Java 语言（或你熟悉的伪代码）编写该类，要求包含以下内容：

私有属性：书名（title）、作者（author）、价格（price）。

构造方法：用于初始化这三个属性。

成员方法：displayInfo(), 用于在控制台打印该图书的详细信息。

1/1

```
public class Book {
    private String title;
    private String author;
    private double price;

    public Book(String title, String author, double price) {
        this.title = (title != null) ? title : "未知书名";
        this.author = (author != null) ? author : "匿名";
        this.price = Math.max(0, price);
    }

    public String getTitle() { return title; }

    @Override
    public String toString() { return "Book {title = '" + title + "', price = " + price + "'}"; }
}
```

2.算法（20）

题目要求：

给定一个已经升序排列的整型数组 `int[] nums = {12, 25, 33, 48, 50, 66, 71, 80};`。

请写出使用二分查找法查找目标值 66 的具体步骤（写出每一轮的 low、high 和 mid 的值）。

请说明二分查找算法的时间复杂度。

```
Find(nums, target):
    mid = nums.length / 2;
    if (nums[mid] == target)
        print("找到了")
    if (nums[mid] < target)
        low = mid
    else
        high = mid
```

复杂度: $O(\log n)$

白盒测试与路径覆盖（10）

假设有以下一段 Java 代码片段，用于处理图书馆逾期欠费的简单逻辑：

```
public void checkFine(int days, boolean isStudent) {
    if (days > 0) { // 语句 1
        if (isStudent) { // 语句 2
            fine = days * 0.5; // 语句 3
        } else {
            fine = days * 1.0; // 语句 4
        }
    } else {
        fine = 0; // 语句 5
    }
    System.out.println("罚金为: " + fine); // 语句 6
}
```

计算环路复杂度：使用环路复杂度公式 $V(G) = E - N + 2$ （其中 E 为边数，N 为节点数）计算该程序的复杂度。

节点数 $N = 5$

边数 $E = 6$

计算: $V(G) = 6 - 5 + 2 = 3$